

GRIMOIRE VAULT · V1.0.0

Getting Started

5-минутный обзор — куда смотреть

Getting Started

5-минутный обзор для тех, кто только зашёл в репозиторий и не уверен, с чего начать.

Что это

Личная база знаний в браузере + Telegram, с шифрованием паролей, семантическим поиском, kanban-доской и shared vaults для семьи.

Live demo: <https://grimoire-vault.vercel.app>

Куда смотреть зависит от того, кто ты



Я хочу пользоваться приложением

→ `USER.md` — гайд пользователя

Что узнаешь:

- Как привязать Telegram-бота
- 13 категорий и теги
- Тriage inbox'a
- Поиск (FTS / Hybrid / Semantic)
- Vim-шорткаты, ⌘K палитра
- Установка PWA на телефон
- Web Push, shared vaults, бэкап

✂ Я разработчик / хочу зафоркать / поднять локально

→ `DEVELOPER.md` — гайд разработчика

Что узнаешь:

- Tech stack и почему выбран
- Локальная установка
- Архитектура (папки, слои)
- Все env vars
- Schema migrations
- API endpoints
- Code conventions
- Performance, security, observability
- Как добавлять новые фичи

Что узнаешь:

- Замысел и эволюция стека
- Подробное описание каждой технологии (Next.js, Supabase, R2, pgvector, transformers.js, ...)
- 21 «волна» разработки в хронологии
- Что для простого юзера, что для продвинутого
- Сравнение с Notion / Obsidian / Logseq / Pocket / Evernote
- Что важно понять о работе с базами данных

Что и когда добавилось

→ CHANGELOG.md — feature timeline

Что обязательно нужно понимать перед работой

О базах данных

Если ты никогда не работал с **PostgreSQL**, прочти короткий обзор в `DEVELOPER.md` → раздел 4.4 о RLS (Row-Level Security). Это **фундамент авторизации** в проекте: не «приложение проверяет, чьи данные читать», а «БД сама не отдаёт чужие строки».

Минимально необходимое:

1. Что такое таблица, столбец, FOREIGN KEY.
2. Как `WHERE` фильтрует строки.
3. Что индекс ускоряет поиск.

Этого хватит на 90% работы.

О Supabase

Supabase — managed Postgres + Auth + Realtime в одном. Он делает три вещи:

1. **Хранит данные** в Postgres (тот же, что ты бы поднял сам).
2. **Авторизует** через email magic-link / password. Выдаёт JWT.
3. **Стримит изменения** через websocket — браузер видит INSERT/UPDATE на других устройствах в реальном времени.

Free tier: 500 MB БД, 2 GB трафика, ∞ запросов. Для personal vault'a этого хватит на годы.

О том, что весь проект — TypeScript end-to-end

Без `tsc --noEmit` не закрывай PR / не деплой. Типы — твой рантайм-сейфти.

```
npx tsc --noEmit    # должно пройти на 0
npx eslint .        # тоже
npx next build      # production build
```

```
# 70 API checks (auth, dedup, export, import, rate limits, ...)
node scripts/backlog-verify.mjs

# 15 Playwright browser checks (UI flows)
npx playwright test
```

Оба должны пройти зелёным перед деплоем.

Pulling the trigger — практический чек-лист

Я хочу попробовать на live версии

1. Открой <https://grimoire-vault.vercel.app/login>
2. Зарегистрируйся email + пароль
3. Открой [/settings](#) → Telegram, выпусти код, привяжи бота [@TheBaseofKnowledge_bot](#)
4. Перешли боту любую ссылку
5. Открой [/inbox](#) — увидишь её там

Я хочу поднять свою копию

1. Сделай форк репозитория (после публикации)
2. Создай Supabase проект
3. Создай R2 bucket
4. Опционально: бот через @BotFather
5. `cp .env.example .env.local`, заполни
6. Прогоните 8 миграций в Supabase SQL Editor
7. `npm install && npm run dev`
8. → <http://localhost:3000>

Подробности — в [DEVELOPER.md](#) → раздел 2.

Я хочу понять, что вообще внутри

1. Открой [PROJECT-STORY.md](#) — narrative обо всём стеке + 21 волна разработки.
2. Прочти 8 миграций в [supabase/migrations/](#) — это **контракт БД**.
3. Посмотри [lib/data/entries.ts](#) — сердце CRUD'a.
4. Посмотри [lib/api-helpers.ts](#) → `withErrorHandler` — паттерн для каждого API-route.

Где задавать вопросы

- Issues в репозитории
- В чате с ботом [@TheBaseofKnowledge_bot](#) (для пользователей живого деплоя)

Эта база знаний — твоя. Если она перестаёт работать на тебя — закрой её честно (*Full export* → *wipe* → возможно, попробуешь *Notion* / *Obsidian*). Лучше иметь *работающий* инструмент чужого, чем заброшенный свой.

GRIMOIRE VAULT · GETTING-STARTED